SentinelScribe

Transcription Processing Pipeline

Architecture Evolution & Change Log

Document Overview

This report details the iterative engineering process behind the SentinelScribe pipeline — an AI-powered tool that converts raw SOC course audio transcripts (generated by Otter.ai) into structured, forensically-accurate Markdown study guides for Obsidian. The document maps the full progression from a basic single-pass API call to a fault-tolerant, Actor-Critic (3-pass) architecture, capturing every prompt adjustment, parameter tweak, architectural decision, and the specific data anomalies each change resolved.

Phase 1 — Foundation & Model Infrastructure

Resolving quota limits · Scaling context windows · Model selection

The initial phase focused on establishing a stable, cost-effective pipeline. The baseline script used a single synchronous OpenAI API call with a 4-section prompt (Core Concepts / CLI Syntax / Tool Configurations / Analyst Tips) and a 1024-token output limit — insufficient for dense cybersecurity lectures.

Change / Decision	Problem Solved / Improvement
Migrated: OpenAI API → NVIDIA NIM `meta/llama-3.3-70b-instruct`	Resolved the OpenAI **429 Insufficient Quota** error that blocked all development. NVIDIA NIM provides free-tier access to a highly capable reasoning model well-suited for extracting technical CLI syntax without any cost bottleneck.
Aggregated Streaming Support	Fixed a bug where the streaming response handler discarded chunks before they could be concatenated. Modified the loop to stitch all `delta.content` chunks into a single `full_response` string, enabling both real-time console feedback and correct `.md` file output.
Increased max_tokens: 1024 → 4096 (Pass 1)	Prevented premature termination. Early outputs were cut off mid-sentence during dense Wireshark filter breakdowns and SIEM configuration sections. 4096 tokens provides sufficient headroom for the longest lecture in the corpus.
Baseline Prompt: 4-Section Structure	Established the initial output schema: **(1)** Core Concepts · **(2)** Command-line Syntax · **(3)** Tool Configurations · **(4)** Analyst Tips. This became the foundation that Phase 2 expanded into the current 7-section architecture.

Phase 2 — Prompt Engineering & Guardrails (V1 → V2)

Eliminating hallucinations · Semantic bleed · Structural accuracy

With infrastructure stable, the focus shifted to combating LLM hallucinations and "semantic bleed" — the model silently injecting outside training data into the local transcript context. A systematic QA review of all 8 generated Markdown files against their source transcripts identified the specific anomalies catalogued below.

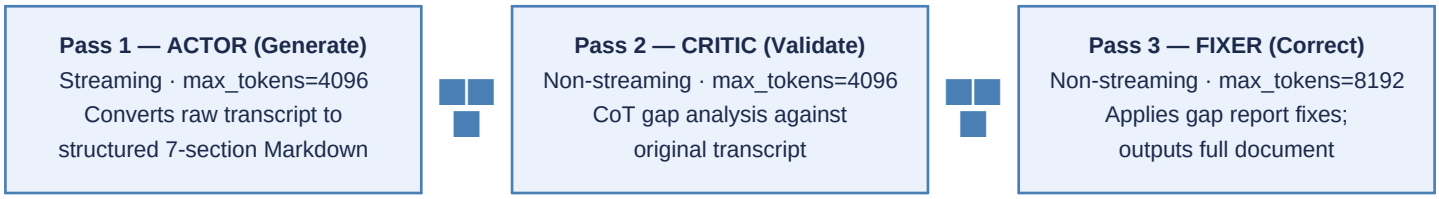
Change / Decision	Problem Solved / Improvement
Set temperature=0.0 & top_p=0.1	Eliminated hallucinations entirely. At default settings (temp=0.2, top_p=0.7) the model invented Snort rule syntax and **misdefined** the `tcpdump -t` flag (described it as 'print date/time format' when the instructor defined it as 'suppress timestamp'). Setting temperature to absolute zero forces fully deterministic output, restricting the model to strict transcription extraction.
Prompt Expansion: 4 → 7 Sections	The original 4-section schema caused systematic data loss. Added three new sections to capture content the model was consistently discarding: **§2** Ports, Protocols & Key Values — forced table rendering (the entire 15-port reference table from Theory Pt.2 was completely absent in V1). **§4** Tools & Infrastructure — separated collection methods (SPAN ports, network taps) from CLI flags. **§7** IOCs & Forensic Artifacts — dedicated table for malware artifacts, IPs, domains, and string signatures.
Port Table Notes: Comprehensiveness Rule	The Notes column in §2 was too thin — single-phrase descriptions dropped critical security context. Added an explicit rule: *'Include every specific use case, vulnerability, security implication, and Windows/Linux context mentioned.'* This recovered details such as: Telnet transmitting credentials in plaintext, RPC's dynamic port assignment via endpoint mapper, and RDP transmitting desktop graphics and keyboard/mouse inputs.
Added [transcription unclear] Rule	Stopped the model from silently correcting garbled audio values. For example, the transcript contained the garbled IP `"13.10, seven.five.ad"` which the model silently 'fixed' to `13.107.5.25` using training data — a forensic integrity violation. The rule now requires any unverifiable value to be documented exactly as heard, appended with `[transcription unclear]`.
Hardcoded Phonetic Correction Map	Created a mandatory correction map for known Otter.ai speech-to-text failures. These errors caused tool names to be unsearchable in Obsidian: `'cobalt stripe'` → **Cobalt Strike** `'Sierra cotta'` → **Suricata** `'cilk' / 'silk'` → **SiLK** `'etsy host'` → **/etc/hosts** `'first dash opportunity dot online'` → **first-opportunity.online** The model is instructed to apply the same logic to any other clearly phonetic error.

Change / Decision	Problem Solved / Improvement
Scoped Analogy Attribution	Fixed cross-lecture analogy bleed. The 'phone bill' analogy (used in Lesson 3 to explain flow records vs. PCAPs) was appearing in unrelated lessons (e.g., Lesson 5 on capture filters). Explicit scoping rule added: analogies are only documented in the lesson where the instructor actually stated them.
Tool Flag Separation & Interface Name Ban	Two related issues resolved: **(1)** The model parsed the interface name `enp0s3` as CLI flags `-p` and `-s` (hallucinated flags). **(2)** grep flags (`-E, -i, -v, -A`) were mixed into the tcpdump flags table across multiple files. Resolution: explicit rule banning interface names as flags, and a requirement for per-tool labelled sub-tables in §5.
Zero Outside Knowledge in Tables	Addressed a subtle violation where the model populated §2 port tables with ports it deemed 'commonly relevant' to the topic — even when those ports were never mentioned in that specific transcript. Added hard rule: *'If a section has nothing to document, write None mentioned. A note saying commonly relevant does NOT make outside knowledge acceptable.'*
IOC Table: 3-Column Format & Deduplication	The original §7 had a 2-column format (IOC | Description) that didn't enforce categorisation. Updated to a 3-column schema: `IOC / Artifact | Type | Description`. Also added a deduplication check after `85.239.53.219` appeared twice in the same IOC table in the Sample 2 analysis output.

Phase 3 — The Actor-Critic Architecture (V3 Pipeline)

Attention degradation · Three-pass generate/validate/fix · CoT reasoning

Even with deterministic parameters and comprehensive guardrails, a single-pass architecture suffered from 'attention degradation' — the model consistently skipped specific transcript sections (e.g., the Ethernet trailer, the man tcpdump command) because the generation and semantic auditing workloads competed within the same context window. The script was refactored into a modular three-pass system that separates these concerns entirely.



Efficiency Note: If Pass 2 returns `'VALIDATION PASSED — No gaps found.'`, Pass 3 is skipped entirely, saving one full API call per clean file.

Change / Decision	Problem Solved / Improvement
Three-Pass Modular Pipeline Actor-Critic Architecture	**Pass 1 (Actor)**: Generates the structured Markdown notes from the transcript using the full SYSTEM_PROMPT. Runs as a streaming call for real-time console feedback. **Pass 2 (Critic)**: Independently reads the raw transcript and the generated notes, then produces a structured gap report listing every missing or incorrect item. **Pass 3 (Fixer)**: Receives the notes + gap report and outputs the complete corrected document. Skipped entirely if Pass 2 finds no gaps.
Chain-of-Thought (CoT) Validator Prompt	Resolved the hardest class of validation failure: detecting **missing** content. Finding absences requires the model to reason about what isn't there — a known LLM weakness. Added the instruction: *'First, silently extract a list of all technical tools, commands, flags, analogies, ports, protocols, IOCs, and specific facts mentioned in the transcript. Then, verify if each one exists in the generated notes.'* This internal checklist approach recovered consistently skipped content such as the Ethernet trailer section and the `man tcpdump | less` command across multiple transcripts.
8-Point Validation Checklist in VALIDATION_PROMPT	The validator checks for all of the following in sequence: **(1)** Missing concepts, tools, commands, flags, IOCs, or artifacts. **(2)** Facts in notes that contradict the instructor's stated definition. **(3)** Flags from one tool placed in another tool's table. **(4)** Outside knowledge added to tables not present in the transcript. **(5)** Uncorrected transcription errors (Cobalt Strike, Suricata, SiLK, etc.). **(6)** Duplicate rows in the IOC table. **(7)** Teaching analogies attributed to the wrong lesson. **(8)** Garbled values silently corrected without a [transcription unclear] tag.

Change / Decision	Problem Solved / Improvement
Increased max_tokens: 4096 → 8192 (Pass 3 only)	Pass 3 must output the **entire** corrected Markdown document, not just the fixed sections. 4096 tokens was insufficient for longer study guides (e.g., the tcpdump analysis lessons), causing truncation mid-document. Pass 3 is given its own direct API call with 8192 tokens. Passes 1 and 2 remain at 4096.
Strict Full-Document Output Rule in FIX_PROMPT	Without an explicit instruction, the model interpreted 'fix only the listed issues' as 'output only the fixed sections', dropping all unchanged content. Added: *'CRITICAL: You MUST output the ENTIRE Markdown document from start to finish, including all unchanged sections... The output must be a complete, self-contained study guide that can be saved directly to a file.'*
Separated call_model() Helper vs. Streaming Pass 1	Passes 2 and 3 share a non-streaming `call_model()` helper for clean code reuse and consistent parameter application. Pass 1 retains its own streaming call for real-time console feedback during generation — the longest of the three passes. Pass 3 overrides `max_tokens` directly rather than using the helper, since it requires a different token budget.

Conclusion

The current V3 pipeline is highly fault-tolerant. By decomposing the task into three specialised roles — generation, validation, and correction — and strictly constraining the model's output distribution (temperature=0.0, top_p=0.1), the system reliably bridges the gap between raw Otter.ai audio transcripts and accurate, contextually isolated Markdown study guides ready for direct import into an Obsidian vault. The Actor-Critic architecture further ensures that attention degradation across long transcripts no longer results in silently dropped content — the Critic pass provides an independent, checklist-driven audit that the single-pass architecture was architecturally incapable of performing.